SPEED TRAINING ZENIKA · ATELIER LLM

# Context Engineering

## *Piloter des livrables avec l'IA*

Un bon résultat vient moins d'un *prompt magique* que d'un **brief clair**, d'un **contexte utile** et de **points de contrôle explicites**.

Dans un contexte métier, l'enjeu n'est pas de "bien parler à l'IA". L'enjeu est d'obtenir plus vite un livrable exploitable: user stories, synthèse d'atelier, plan d'action, analyse d'impact, plan de release.

# Le principe de l’atelier

L’objectif n’est pas de collectionner des recettes ou des phrases magiques. L’objectif est d’apprendre à cadrer une demande, fournir le bon contexte, puis relire la sortie avec le bon niveau d’exigence.

## Passer du réflexe conversationnel au réflexe d’ingénierie

| Réflexe conversationnel | Réflexe de pilotage |
| --- | --- |
| "fais-moi ça" | objectif explicite |
| "non, pas comme ça" | brief structuré |
| "recommence" | critères de qualité |
| "tu as oublié X" | bon contexte métier |
| "mais pourquoi ?" | contrôle / reprise |

# Vibe coding vs AI-driven dev

| | Vibe coding | AI-driven dev |
|---|---|---|
| Logique | explorer vite | produire de façon fiable |
| Marche bien quand | on cherche des idées, une première structure, un brouillon, un prototype | on veut un livrable réutilisable, partageable, relisible |
| Exemples | tester 3 formulations de user story, esquisser un plan, faire émerger des pistes | consolider un brief, préparer un support de release, extraire décisions, risques et actions |
| Risque accepté | un peu de flou, de variance, de déchet | peu de flou, besoin de cohérence et de traçabilité |
| Réflexe | on tente, on observe, on jette si besoin | on cadre, on contrôle, on capitalise |

Les deux approches sont utiles. Le bon réflexe est de viber pour explorer, puis de piloter pour livrer.

# Le déroulé

## 4 blocs

1. **Comprendre pourquoi l'IA varie** — sentir ce qui change d'une réponse à l'autre
2. **Structurer un brief utile** — transformer une demande floue en demande exploitable
3. **Fournir le bon contexte** — donner les bons éléments, pas tout le bruit autour
4. **Relire et capitaliser** — transformer une session en décisions, risques et actions

## Rythme

Chaque bloc suit le même format: une démo courte, puis un exercice pratique. Le but n'est pas de devenir technique. Le but est d'acquérir des réflexes simples pour mieux cadrer, mieux vérifier et mieux réutiliser les sorties de l'IA.

```
14h30  cadrage + repères de base
14h45  démo 1 + exercice 1   variabilité des réponses
15h20  démo 2 + exercice 2   brief structuré
16h05  pause + débrief
16h25  démo 3 + exercice 3   contexte utile
```

# Ce qu’on refuse

## On refuse l’accumulation confuse

Toujours plus de contexte, toujours plus d’outils, toujours plus d’itérations. Cela peut donner des résultats impressionnants, mais les sessions deviennent chères, opaques et difficiles à relire.

Dans un contexte métier, le risque est simple: on obtient un résultat qui a l’air crédible, mais dont personne ne sait vraiment expliquer l’origine, les limites ou les arbitrages.

## On privilégie une approche pilotée

On ne cherche pas à discuter avec l’IA jusqu’à tomber sur une bonne réponse. On cherche à préparer une entrée claire pour obtenir une sortie utile, vérifiable et réutilisable.

Cette approche demande de ralentir un peu: poser le problème, choisir ce qu’on donne au modèle, vérifier les sorties intermédiaires, puis repartir proprement quand la session dérive.

| On refuse | On privilégie |
| --- | --- |

# 3 repères pour bien travailler avec l'IA

Pour aller plus loin sur ce point: voir **Handout H1 — Fondamentaux : comment fonctionne réellement un LLM.**

## 1. La réponse peut varier

Le même besoin peut produire plusieurs réponses correctes, moyennes ou hors sujet. Une réponse n'est donc pas une vérité: c'est une proposition.

## 2. Le flou coûte cher

Si la demande est vague, l'IA comble les trous comme elle peut. Plus il y a de flou, plus il y a de variance, d'oublis et de reformulations hasardeuses.

## 3. Le contexte pèse sur la sortie

Les éléments qu'on donne au modèle influencent fortement la réponse: objectifs, contraintes, documents, vocabulaire, exemples, historique de session.

Ce qui aide vraiment n'est pas de donner plus d'information. C'est de donner **la bonne information**, au bon moment, dans une forme exploitable.

Démo + exercice 1

# Ce que ça change pour nous

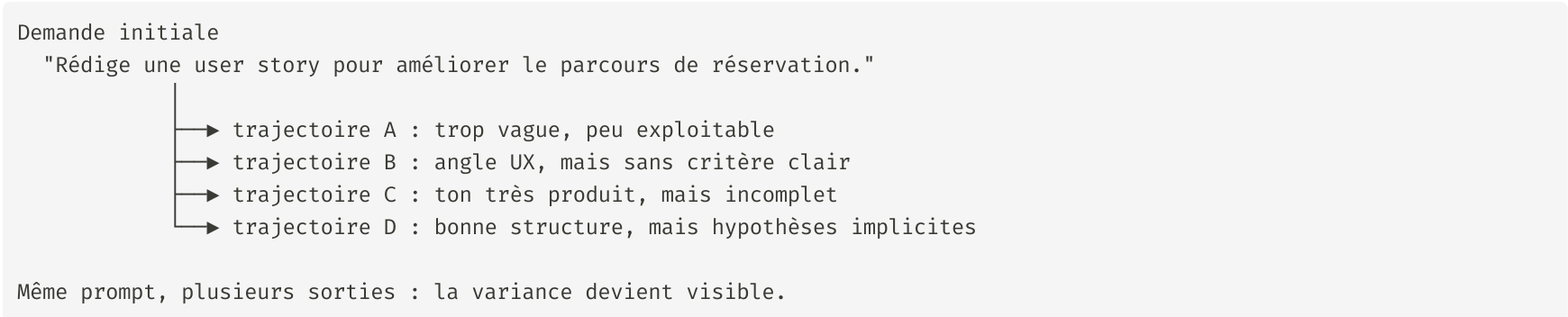
Si l'IA varie beaucoup, il ne faut pas seulement corriger la sortie. Il faut souvent mieux cadrer l'entrée.

Le premier réflexe utile est donc empirique: relancer, comparer, identifier ce qui reste stable, puis réduire le flou avant de demander une version finale.



# Démo 1 — Une même demande, plusieurs résultats

On part d'une demande métier volontairement simple, puis on change très peu de mots. Le but n'est pas encore de produire un bon livrable. Le but est de sentir que chaque mot déplace le résultat.




Ensuite on ajoute `pour mobile`, `avec contrainte de délai`, `pour les clients premium`, `avec critères d'acceptation`. La réponse change vite, parfois dans le bon sens, parfois en introduisant de nouvelles hypothèses.

# Exercice 1 — Explorer

## À faire

- choisir un prompt court
- régénérer 5 fois
- noter ce qui reste stable
- changer 3 mots maximum
- ajouter une contrainte métier: délai, cible, canal, format attendu
- comparer une demande floue et une demande structurée

## À observer

Cherchez les mots qui resserrent la sortie et ceux qui la contaminent. Une bonne intuition du modèle se construit en manipulant de petites différences, pas en lançant directement une grosse tâche.

```
prompt robuste
    ├── regenerate 1 : même structure
    ├── regenerate 2 : même structure
```

# Pourquoi 3 mots changent tout

Le modèle produit sa réponse **mot après mot**.

Chaque mot déjà présent dans la demande influence la probabilité des mots suivants.

Donc :

- certains mots ont **peu d'effet**
- d'autres mots réorientent fortement la suite
- un brief plus précis resserre la trajectoire

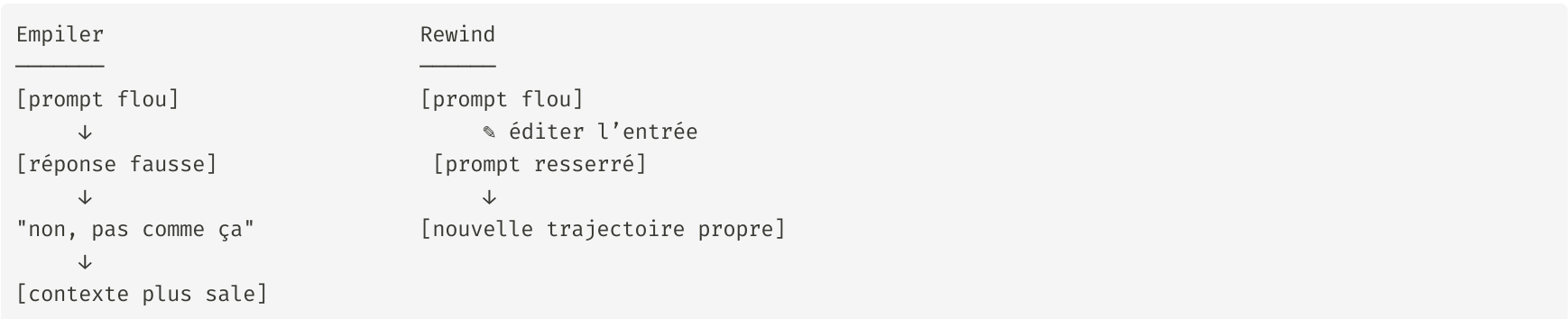
`mobile`, `premium`, `urgence`, `critères d'acceptation`, `sans jargon`, `en 5 points`

Ces mots agissent comme des **signaux** : ils rendent certaines suites plus probables, et d'autres moins probables.

> Pour le détail théorique sur les tokens, la variance et le contexte: voir **Handout H1 — Fondamentaux : comment fonctionne réellement un LLM.**

# Le signal de rewind

Quand la première réponse est mauvaise, la tentation est de corriger par-dessus. Mais chaque correction ajoute de nouveaux tokens dans une fenêtre déjà mal orientée.




Le réflexe d'ingénieur : traiter la mauvaise sortie comme un message de compilateur. On revient à l'entrée, on la corrige, puis on relance proprement.

Démo + exercice 2

# Du besoin flou au brief structuré

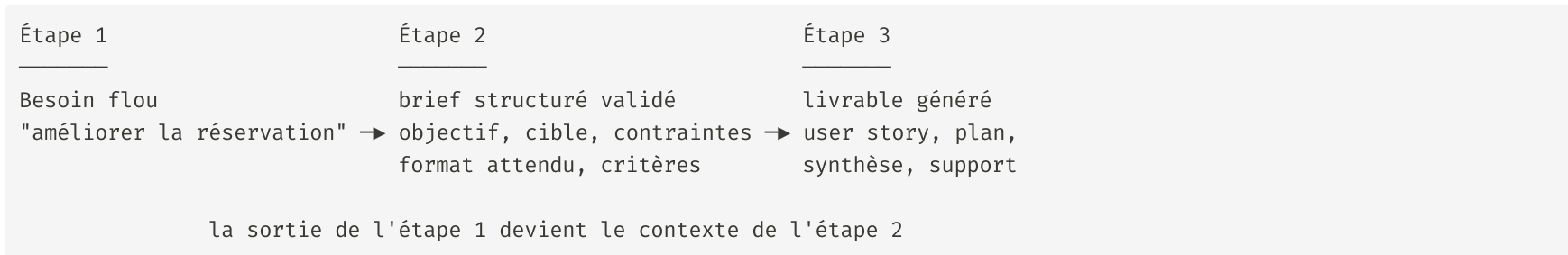
Du besoin exprimé à l'oral vers une structure claire, puis vers un livrable.

Après l'exploration, le problème devient clair : le langage naturel laisse beaucoup d'ambiguïtés. On va donc passer par un brief structuré qui rend les attentes visibles avant de demander une sortie finale.



# Le principe du trampoline

Le format intermédiaire n'est pas magique. Il fonctionne parce qu'il rend l'intention plus dense, plus vérifiable, et parce qu'une première sortie validée sert ensuite de base solide pour la suite.




On construit le résultat étage par étage. Au lieu de demander au modèle de tout deviner d'un coup, on lui fait produire une structure, on la vérifie, puis on s'en sert comme fondation.

# Démo 2 — Structurer un brief utile

On demande d’abord au modèle de proposer une structure de brief. Ensuite seulement, on lui demande de produire un livrable: user story, synthèse, plan d’action, note ou support.

```
objectif: réduire les abandons sur mobile
public_cible: clients qui réservent depuis leur téléphone
contrainte: amélioration visible avant la prochaine release
format_attendu: user story avec critères d'acceptation
points_de_vigilance:
  - ne pas inventer de dépendance technique
  - rester compréhensible par l'équipe produit
```

La structure sert de contrat temporaire. Elle retire de la prose inutile, rend les champs visibles, et permet à l’humain de valider l’intention avant de demander une sortie plus engageante.

# Du prototype au prompt consolidé

La démo ne s'arrête pas au premier livrable acceptable. Le vrai actif est un brief réutilisable, testé dans un chat vierge, qui reproduit un niveau de qualité correct sur plusieurs sujets.




Quand le même brief produit une sortie robuste sur deux sujets différents, on a fabriqué un actif réutilisable: dans un chat, un outil, un atelier ou un rituel d'équipe.

# Mission

Créer un petit livrable à partir d'un brief structuré. Le domaine doit rester métier: backlog, parcours client, synthèse d'atelier, plan d'action, communication de release.

# But

Construire quelque chose de manière structurée. Le brief final doit contenir la structure intermédiaire, les règles de sortie et au moins un exemple du format attendu.

```
idée métier
    ↓
brief structuré : objectif / cible / contraintes / format / critères
    ↓
livrable : user story / synthèse / plan / note / support
    ↓
brief consolidé testé dans un chat vide
```

*Débrief : montrez surtout le brief consolidé, pas seulement le rendu.*

Pour approfondir cette étape: voir Handout H3 — Exo 2 : clarifier un besoin avec un brief structuré.

# Pause + débrief

Deux questions à poser à la salle.

1. Qu’est-ce que la formalisation a résolu par rapport au regenerate ?
2. Qu’est-ce qu’elle n’a pas résolu ?

| Ce que le trampoline règle | Ce qu’il ne règle pas |
| --- | --- |
| ambiguïté locale | mauvais objectif |
| format instable | mauvaise architecture |
| champs invisibles | données confidentielles |
| variance inutile | session agentique sale |

La deuxième partie change d’échelle: on ne formalise plus seulement une demande. On apprend à préparer un **contexte utile** pour une tâche plus large.

# Le contexte réellement pris en compte




Context window — entrée effective du modèle · 120 000 token window

Historique utile de la conversation: objectif, clarifications, corrections et décisions déjà prises.

C’est tout ce que le modèle prend réellement en compte quand il répond.

Ce contexte contient parfois bien plus que le dernier message utilisateur: historique, consignes, documents joints, sorties précédentes, résultats d’outils, éléments relus ou copiés plus tôt dans la session.

- Le modèle ne "voit" pas seulement votre dernière demande.
- Chaque élément ajouté peut aider, brouiller ou détourner la réponse.
- Piloter l’IA commence par choisir ce qui mérite vraiment d’entrer.

Piloter l’IA, c’est surtout choisir ce qui entre dans ce contexte, et savoir quand repartir d’une base propre.

# Constellation 1 — session simple




Session simple et concentrée · peu de bruit, repères clairs

> Le besoin est explicite: ce qu'on veut obtenir, pour qui, et dans quel contexte.

Une session simple n'est pas forcément pauvre. Elle est lisible : objectif, format attendu, quelques contraintes, un exemple, puis la demande.

Lisez-la de gauche à droite : cadre, objectif, contraintes, exemple, puis demande finale.

- La simplicité vient de la cohérence, pas du manque d'information.
- Les signaux importants sont rares, proches et non contradictoires.
- Le modèle a peu d'occasions de partir ailleurs.

Ici, l'attention est concentrée. Le modèle n'a pas beaucoup d'occasions de partir ailleurs, parce que les repères utiles sont peu nombreux et cohérents.

# Constellation 2 — session chargée




Session chargée · trop d’éléments s’accumulent dans le contexte

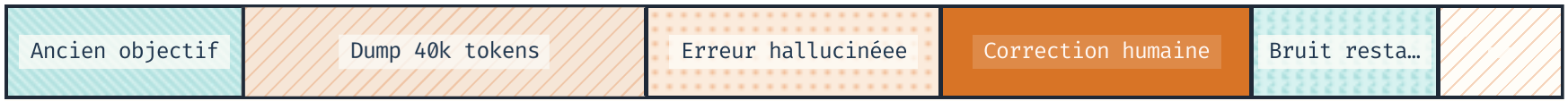
La session garde beaucoup de messages, de corrections et de décisions passées.

Une session peut se charger progressivement sans être polluée pour autant. On garde l’historique, on ajoute des documents, on compare plusieurs essais, on corrige au fil de l’eau.

- Le contexte devient plus riche, mais aussi plus lourd.
- Certains éléments anciens peuvent prendre trop de place.
- Une session trop chargée ralentit aussi le modèle: il doit traiter plus d’éléments avant de répondre.
- La bonne question est: tout ce qui reste ici aide-t-il encore la tâche du jour ?

Le risque n’est pas seulement l’erreur. Le risque, c’est la dérive douce: la session reste crédible, mais s’éloigne progressivement du besoin initial.

# Constellation 3 — session polluée




Fenêtre polluée — signaux utiles noyés dans le bruit · 180 000 token window

Un gros bloc brut noie le reste du signal et dilue l’attention.

Une session polluée contient souvent de bonnes informations, mais aussi des objectifs périmés, des erreurs déjà corrigées, des copies trop longues et des contradictions invisibles.

- Une session polluée contient souvent de bonnes informations et trop de bruit.
- Corriger par-dessus peut aggraver le problème.
- Le bon geste est parfois de repartir proprement.

Quand la session dérive, continuer à corriger peut aggraver le contexte. Le bon geste est souvent de revenir en arrière, d’éditer l’entrée, ou de repartir d’une base propre.

# Constellation 4 — session préparée




Session préparée et guidée · repères clairs avant de lancer la demande

Une vue courte de la tâche: sujet, objectif, documents utiles et limites à respecter.

Une bonne session peut être préparée avant même de poser la demande. On rassemble d'abord les bons repères, puis on lance le travail.

- On ne met pas tout: on choisit ce qui aide vraiment.
- Les documents utiles, règles stables et exemples servent de repères.
- Cette préparation aide aussi l'humain à voir ce qui manque ou ce qui reste flou.

Ce type de session n'essaie pas d'être complet. Il cherche surtout à être clair: peu de bruit, des repères solides, et une demande finale mieux cadrée.

Démo + exercice 3

# Préparer un contexte utile

Un bon contexte est dense en signes utiles et pauvre en bruit.

Elle contient des tokens importants — noms de fichiers, commandes, invariants, décisions — et des relations claires entre eux. Elle n'essaie pas de tout savoir : elle prépare le modèle pour la tâche du jour.



# Visualiser la context window

La `context window`, c'est tout ce qui a réellement été donné au modèle au moment où il répond: les consignes, l'historique, les documents ajoutés, les essais précédents et les corrections.

- Cela permet de voir ce qui pèse vraiment sur la réponse.
- Si l'entrée est floue ou trop chargée, la sortie le sera souvent aussi.
- Relire ce contexte aide à reprendre la main.
- Dans des agents comme `Codex` ou `Claude`, on peut souvent en retrouver des traces dans l'historique, les fichiers ajoutés, les résultats d'outils et les résumés de session.

Visualiser la `context window` permet d'expliquer les écarts, de retirer le bruit inutile, puis de repartir sur une base plus claire.

# Préparer une note de cadrage pour l’IA

Avant de demander un livrable, on prépare une courte note qui explique à l’IA sur quoi travailler, avec quelles limites, et avec quels repères.




Contexte de travail préparé · repères utiles avant la demande finale

La tâche à réaliser est formulée clairement dès le départ.

- Ce n’est pas un document complet: c’est un cadrage de travail.
- On y met surtout: objectif, documents utiles, règles stables, points de contrôle et demande finale.
- Un document court, relu, vaut mieux qu’un gros bloc vague.
- Les bons repères évitent à l’IA de partir dans tous les sens.

`L'agent peut` proposer une première structure, regrouper les informations utiles et signaler des manques visibles.

# Exercice 3 — Cartographier

Générez une note de cadrage ciblée pour un sujet métier précis.

## 1. Cibler

Choisir un sujet concret : clarifier un parcours client, préparer une release, cadrer une évolution produit, synthétiser un atelier.

## 2. Générer

Faire produire une note de cadrage avec : objectif, acteurs, règles métier, dépendances, zones floues et points de vigilance.

## 3. Lire et corriger

Relire de près, retirer le bruit, corriger les approximations, compléter les manques, puis seulement lancer le vrai travail.

```
session propre + sujet clair
          ↓
note de cadrage générée
          ↓
lecture humaine + corrections
          ↓
nouvelle session + note chargée
          ↓
réponse plus directe, moins d'exploration inutile
```

Test de vérité : nouvelle session, même note chargée, même sujet.

Démo + exercice 4

# Extraire du contexte utile

Une session imparfaite n’est pas un déchet. C’est une matière à exploiter.

Une session longue contient souvent des éléments précieux: décisions prises, points flous, objections, hypothèses, risques, reformulations utiles. Ce travail a déjà été produit. Il peut être extrait, relu, puis réutilisé.



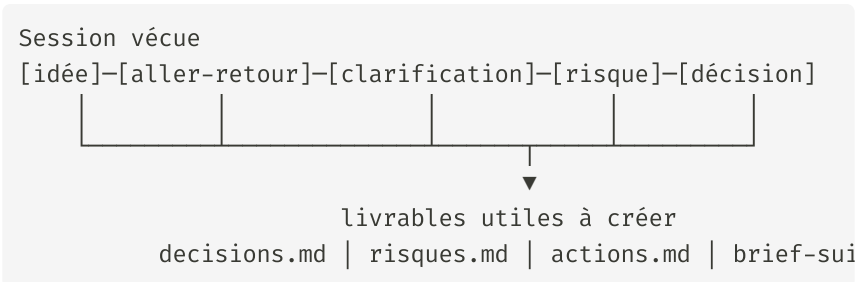
# Exploiter les traces

## Ce qu’on extrait

- décisions prises
- risques détectés
- questions ouvertes
- actions à lancer
- règles ou formulations réutilisables
- synthèse propre

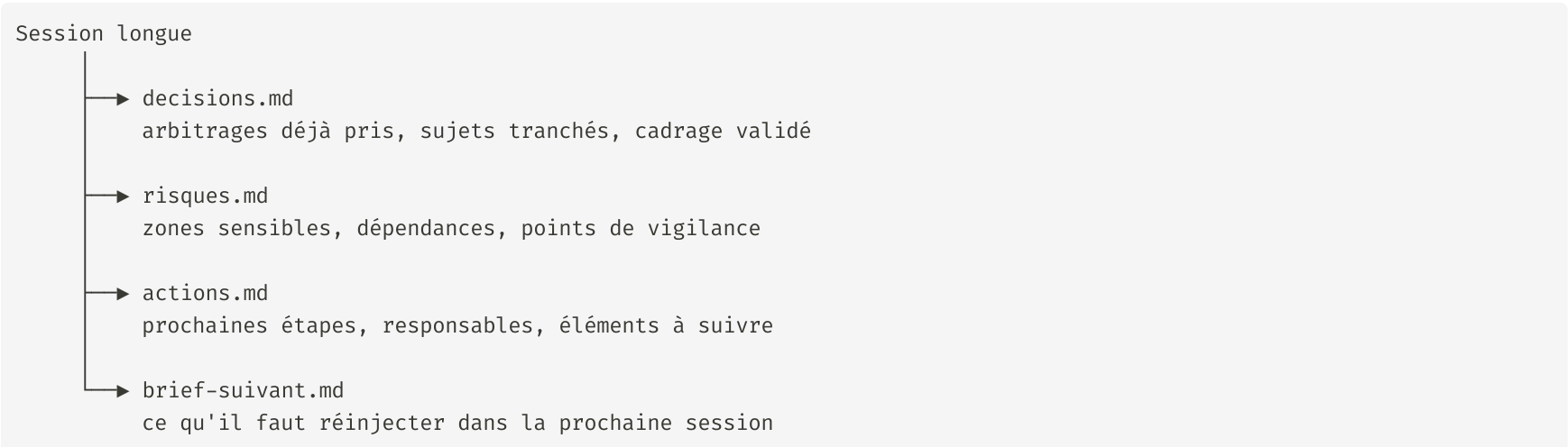
## Le réflexe

Si la session patine, le bon réflexe n’est pas seulement de corriger tour après tour. Il faut parfois récupérer ce qui a de la valeur, puis repartir proprement.



# Recettes d’extraction

Le prompt d’extraction doit viser un livrable précis. `Résume cette session` est moins utile que `extrais les décisions, risques, questions ouvertes et prochaines actions`.




L’ancienne session peut ensuite être laissée de côté. L’essentiel a été extrait, relu et transformé en matière réutilisable.

# Exercice 4 — Extraire

Prenez une session existante. Repérez ce qui mérite d'être gardé. Créez un livrable utile.

## Identifier

Où sont les décisions, les risques, les malentendus, les questions encore ouvertes, les points utiles pour la suite ?

## Extraire

Demander une extraction ciblée: décisions prises, risques détectés, questions ouvertes, actions à lancer, ou note à réinjecter dans une prochaine session.

## Relire

Élaguer, corriger, densifier. Ne jamais transformer une extraction non relue en source d'autorité.

```
ancienne-session-longue
```

# La boucle complète

expérimenter → formaliser → cartographier → extraire

```
1. Accepter que la sortie puisse varier
       ↓
2. Clarifier le besoin avant de demander le livrable
       ↓
3. Préparer un bon contexte pour la tâche
       ↓
4. Récupérer les éléments utiles d'une session
       ↓
5. Repartir avec une base plus claire
```

À chaque étape, le même geste revient : choisir ce qu'on donne au modèle, rendre les liens plus clairs, puis relire avec un oeil humain. C'est ce qui transforme une conversation floue en travail piloté.

# 5 réflexes pour lundi matin

1. **Tester plusieurs sorties** avant de croire qu'une formulation est solide.
2. **Montrer un format attendu** plutôt que décrire longuement le résultat.
3. **Repartir proprement** quand la session dérive, au lieu de corriger sans fin.
4. **Préparer une note de cadrage** avant de lancer une tâche large.
5. **Capitaliser ce qui compte** avant de fermer une session.

```
moins de flou       plus de cadrage
moins de volume     plus de repères utiles
moins de dispersion → plus de contrôle
```

# Questions

Qu’est-ce que vous allez changer dans votre prochaine session LLM ?

On ne cherche pas à devenir plus bavard avec le modèle. On cherche à mieux cadrer ce qu’on lui donne, ce qu’on attend, et ce qu’on garde pour la suite.